<쿠버네티스(kubernetes) 설치>

여기서는 쿠버네티스 종류 중에서 가벼운 MiniKube를 설치

1. 패키지 목록 새로고침, curl 패키지 설치 (cf. curl: 데이터를 전송 또는 받아올 때 사용)

sudo apt update && sudo apt install -y curl

2. MiniKube 실행파일을 인터넷에서 다운로드 및 실행

curl -LO
https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64

sudo install minikube-linux-amd64 /usr/local/bin/minikube

3. 시작하기 (Docker 드라이버 사용)

// 도커에게 쿠버네티스 서버 역할을 할 컨테이너 생성명령. sudo docker ps -a로 확인

minikube start —driver=docker // 처음 실행할 때, 다시 실행할 때는 minikube start

4. kubectl 설치

sudo apt install -y kubectl

5. 클러스터 확인

// cf. 파드(Pod): 쿠버네티스에 실행되는 애플리케이션이 가장 작은 배포 단위

// 파드(Pod)는 하나 이상의 컨테이너를 포함

kubectl get nodes

```
hgahn@ads:~$ kubectl get nodes
NAME       STATUS   ROLES           AGE   VERSION
minikube   Ready    control-plane   16h   v1.34.0
```

kubectl get pods -A // 클러스터의 모든 네임스페이스에 있는 파드(Pod)를 보여줌

```
hgahn@ads:~$ kubectl get pods -A
NAMESPACE     NAME                                READY   STATUS    RESTARTS       AGE
default       tomcat-deployment-668b6d8d6d-s7wn9  1/1     Running   1 (108m ago)   16h
kube-system   coredns-66bc5c9577-jh7lz            1/1     Running   1 (108m ago)   16h
kube-system   etcd-minikube                       1/1     Running   1 (108m ago)   16h
kube-system   kube-apiserver-minikube             1/1     Running   1 (108m ago)   16h
kube-system   kube-controller-manager-minikube    1/1     Running   1 (108m ago)   16h
kube-system   kube-proxy-6bfj4                    1/1     Running   1 (108m ago)   16h
kube-system   kube-scheduler-minikube             1/1     Running   1 (108m ago)   16h
kube-system   storage-provisioner                 1/1     Running   3 (106m ago)   16h
```

<쿠버네티스가 운영되는 흐름 간단한 예시>

1. **Docker 이미지 → Deployment로 관리**
   - 컨테이너 자동 재시작
   - 레플리카(확장) 가능

- 롤링 업데이트 가능

2. **Service(NodePort/Cluster/LB)**
   - 외부 접근 포트 지정 → Tomcat, MariaDB, Jenkins 접속 가능
3. **PVC로 MariaDB 데이터 저장**
   - PVC: Pod는 휘발성이기 때문에 Pod가 삭제시 데이터는 남도록 하기 위해 PVC를 사용
   - 컨테이너가 재시작되어도 데이터 유지

<쿠버네티스용 tomcat 컨테이너 생성 예시>

1. **사전 설치 : docker 설치**

2. 이미지 다운로드

sudo docker pull tomcat:9.0 // docker hub에서 이미지 다운로드

3. 빌드 결과 확인

sudo docker images // 목록에서 my-custom-tomcat 이미지 확인

4. Deployment YAML

// 역할
- Pod(컨테이너) 생성과 관리를 담당
- 몇 개의 Pod를 띄울지(replicas) 설정 가능
- Pod가 죽으면 자동으로 다시 만들어서 항상 원하는 상태를 유지
- 이미지, 환경변수, 포트, readiness(로드밸런싱)/liveness(재시작) probe 등 컨테이너 세부 설정 포함
- 어떤 컨테이너를 몇 개 만들고, 어떻게 관리할지 알려주는 설계도

5. Service YAML

// 역할
- Pod에 접근할 수 있는 통로를 만드는 역할
- Pod IP는 동적으로 바뀌기 때문에 직접 접속이 어려움
- Service가 고정된 IP/포트를 제공하여 외부/내부에서 접근 가능
- 여러 Pod를 로드밸런싱 가능 (복제된 Pod가 여러 개 있을 때)
- 항상 연결할 수 있는 문을 만들어주는 역할

6. Deployment YAML 예시

tomcat-deployment.yaml // k8s 폴더를 만들고 vi tomcat-deployment.yaml 저장

```
--------------------------------
apiVersion: apps/v1
kind: Deployment
metadata:
  name: tomcat-deployment
```

```
spec:
  replicas: 1
  selector:
    matchLabels:
      app: tomcat
  template:
    metadata:
      labels:
        app: tomcat
    spec:
      containers:
        - name: tomcat
          image: tomcat:9.0
          ports:
            - containerPort: 8080
```
---

7. Service YAML 예시

tomcat-service.yaml // k8s 폴더를 만들고 vi tomcat-service.yaml 저장

```
apiVersion: v1
kind: Service
metadata:
  name: tomcat-service
spec:
  type: NodePort
  selector:
    app: tomcat
  ports:
    - protocol: TCP
      port: 8080          # Service 내부 포트
      targetPort: 8080    # 컨테이너 포트
      nodePort: 30080     # 외부 접속용 포트
```

<적용 순서>

1. Deployment 삭제 후 재적용 (이미 존재하는 경우)

```
kubectl delete deployment tomcat-deployment
kubectl apply -f tomcat-deployment.yaml
kubectl apply -f tomcat-service.yaml
```

2. Pod이름 확인

kubectl get pods

NAME	READY	STATUS	RESTARTS	AGE
tomcat-deployment-668b6d8d6d-s7wn9	1/1	Running	0	10m

3. 웹앱 ROOT 배포 후 접속 확인

// ROOT.war 복사 --> 여기서 <pod-name>은 위에서 NAME 이름을 의미함

kubectl cp /mnt/c/ROOT.war <pod-name>:/usr/local/tomcat/webapps/ROOT.war

// 예시

kubectl cp /mnt/c/ROOT.war tomcat-deployment-668b6d8d6d-s7wn9:/usr/local/tomcat/webapps/ROOT.war

```
hgahn@ads:~$ cd /
hgahn@ads:/$ ls
Docker  bin.usr-is-merged  dev  home  lib               lib64       media  opt   root  sbin               snap  sys  usr
bin     boot               etc  init  lib.usr-is-merged  lost+found  mnt    proc  run   sbin.usr-is-merged  srv   tmp  var
hgahn@ads:/$ cd mnt
hgahn@ads:/mnt$ ls
c  wsl  wslg
hgahn@ads:/mnt$ cd c
hgahn@ads:/mnt/c$ ls
ls: cannot access 'DumpStack.log.tmp': Permission denied
ls: cannot access 'hiberfil.sys': Permission denied
ls: cannot access 'pagefile.sys': Permission denied
ls: cannot access 'swapfile.sys': Permission denied
'$Recycle.Bin'              Intel            'Program Files (x86)'        User_manual   hiberfil.sys                  pagefile.sys
 99.참고자료                Logs              ProgramData                 Users         inetpub                       recovery
'Documents and Settings'    OneDriveTemp      Python310                   Windows       jdk11                         sts4
 DumpStack.log              PerfLogs         'System Volume Information'  aiproject     logUploaderSettings.ini       swapfile.sys
 DumpStack.log.tmp         'Program Files'    Temp                        aisource      logUploaderSettings_temp.ini  ydataProfiling
hgahn@ads:/mnt/c$ kubectl cp /mnt/c/ROOT.war tomcat-deployment-668b6d8d6d-s7wn9:/usr/local/tomcat/webapps/ROOT.war
```

// tomcat 컨테이너 진입

kubectl exec -it <pod-name> -- ls /usr/local/tomcat/webapps

// alias 만들기 (pod-name이 길어서 컨테이너 접근시 용이하게 하기 위함)

alias kextomcat='kubectl exec -it $(kubectl get pods -l app=tomcat -o jsonpath="{.items[0].metadata.name}") -- bash'

kextomcat

```
hgahn@ads:~$ alias kextomcat='kubectl exec -it $(kubectl get pods -l app=tomcat -o jsonpath="{.items[0].metadata.name}") -- bash'
hgahn@ads:~$ kextomcat
root@tomcat-deployment-668b6d8d6d-s7wn9:/usr/local/tomcat# pwd
/usr/local/tomcat
root@tomcat-deployment-668b6d8d6d-s7wn9:/usr/local/tomcat# ls
bin           conf            filtered-KEYS  LICENSE  native-jni-lib  README.md      RUNNING.txt  upstream-KEYS  webapps.dist
BUILDING.txt  CONTRIBUTING.md  lib            logs     NOTICE          RELEASE-NOTES  temp         webapps        work
root@tomcat-deployment-668b6d8d6d-s7wn9:/usr/local/tomcat# cd webapps
root@tomcat-deployment-668b6d8d6d-s7wn9:/usr/local/tomcat/webapps# ls
root@tomcat-deployment-668b6d8d6d-s7wn9:/usr/local/tomcat/webapps# exit
exit
```

4. 브라우저 접속

kubectl port-forward service/tomcat-service 30080:8080 & // & ==> 터미널 하나로 쓰고 싶을 때 백그라운드 실행

http://localhost:30080

5. 기타

클러스터(Cluster) : 쿠버네티스 전체 시스템을 의미
노드(Node) : 클러스터 안에 있는 서버 1대
파드(Pod) : 쿠버네티스에서 배포되는 최소 단위
컨테이너(Container) : 애플리케이션을 실행하기 위한 최소 실행 단위

minikube status // 쿠버네티스 상태
minikube start // 쿠버네티스 시작
minikube stop // 쿠버네티스 종료
kubectl get nodes // 현재 클러스터에 등록된 노드들의 상태 확인
kubectl get pods //pod 이름 확인
kubectl describe pod <pod-name> // Pod 상세 정보에서 컨테이너 확인
kubectl exec -it <pod-name> -c tomcat -- /bin/bash // tomcat 컨테이너 접근
kubectl exec -it <pod-name> -c mariadb -- /bin/bash // mariadb 컨테이너 접근
kubectl logs <tomcat-pod-name> // tomcat 로그 확인
kubectl get svc tomcat-service // NodePort 연결 확인 (ex. 8080:30080 => 내부 서비스포트:외부 호스트포트)

```
hgahn@ads:~$ kubectl get svc tomcat-service
NAME            TYPE       CLUSTER-IP     EXTERNAL-IP   PORT(S)          AGE
tomcat-service  NodePort   10.109.205.1   <none>        8080:30080/TCP   18h
```

kubectl port-forward service/tomcat-service 30080:8080 & // & ==> 터미널 하나로 쓰고 싶을 때 백그라운드 실행

minikube status // Minikube 상태 확인

```
hgahn@ads:~$ minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured
```

minikube ip // Minikube 환경에서 가상머신 또는 노드의 IP 주소 확인